

IMPERIAL

Adaptive Resource Deployment for Cyber Defence

Better Protection Through Multi-Agent Reinforcement Learning

Miracle Akanmode

Project Defense: 16/09/2025

Supervisor: Kelly Zhang

Agenda

- 1 Introduction
- 2 Problem Statement
- 3 Our Solution
- 4 Methodology
- 5 Results
- 6 Discussion and Conclusion

Introduction

What This Research is About

Research Overview

Problem: Static cyber defenses fail against adaptive adversaries

Approach: Multi-agent reinforcement learning for adaptive deception

Solution: Policy Optimization based agent learning for optimal deceptive resource placement

Result: Up to **80% performance improvement** over baselines



Ancient War Strategy Quotes

Sun Tzu Quotes from the Art of War

“All warfare is based on deception.”

“He will win who knows when to fight and when not to fight.”

“The greatest victory is that which requires no battle.”

— Sun Tzu, The Art of War.

These quotes support our approach: creating uncertainty and choosing when to act are central to the defense agent’s strategy.

Reinforcement Learning in Cyber Defence

Key Terms and Concepts

Honeypot / Decoy — a fake asset deployed to lure attackers and gather signals.

Asset / Critical asset — real systems we want to protect (databases, servers).

DMZ / Subnet — parts of the network (DMZ = public-facing zone).

Reconnaissance — attacker scanning and discovery activity.

Lateral movement — attacker moving from one compromised host to others.

Exploit / Technique — concrete attacker action to gain access or escalate.

APT (Advanced Persistent Threat) — long-term, sophisticated attacker campaign.

MITRE ATT&CK — standardized vocabulary of attacker tactics & techniques (used to model red).

IOC (Indicator of Compromise) / Telemetry — alerts and signals from hosts and network.

SIEM — system that aggregates security alerts and logs for detection/analysis.

Return / Cumulative reward — total (discounted) reward an agent accumulates (objective to maximize).

Discount factor (γ) — how much the agent values future rewards vs immediate ones.

Advantage / GAE — estimate of how much better an action was than expected (used to train PPO).

Actor-critic — architecture with separate policy (actor) and value (critic) networks.

Exploration vs Exploitation — testing new actions vs using known good actions.

Problem Statement

The Cybersecurity Challenge

Problem Statement

The Cybersecurity Challenge

Alert Fatigue: 4,000+ daily alerts, 67% get Ignored

Asymmetric Warfare: Attackers need just one success, while defenders must protect everything, everytime

Static Defenses: Predictable patterns don't stand a chance

Base-Rate Fallacy: Many real threats get missed

Attacker

Need 1 success

VS

Defender

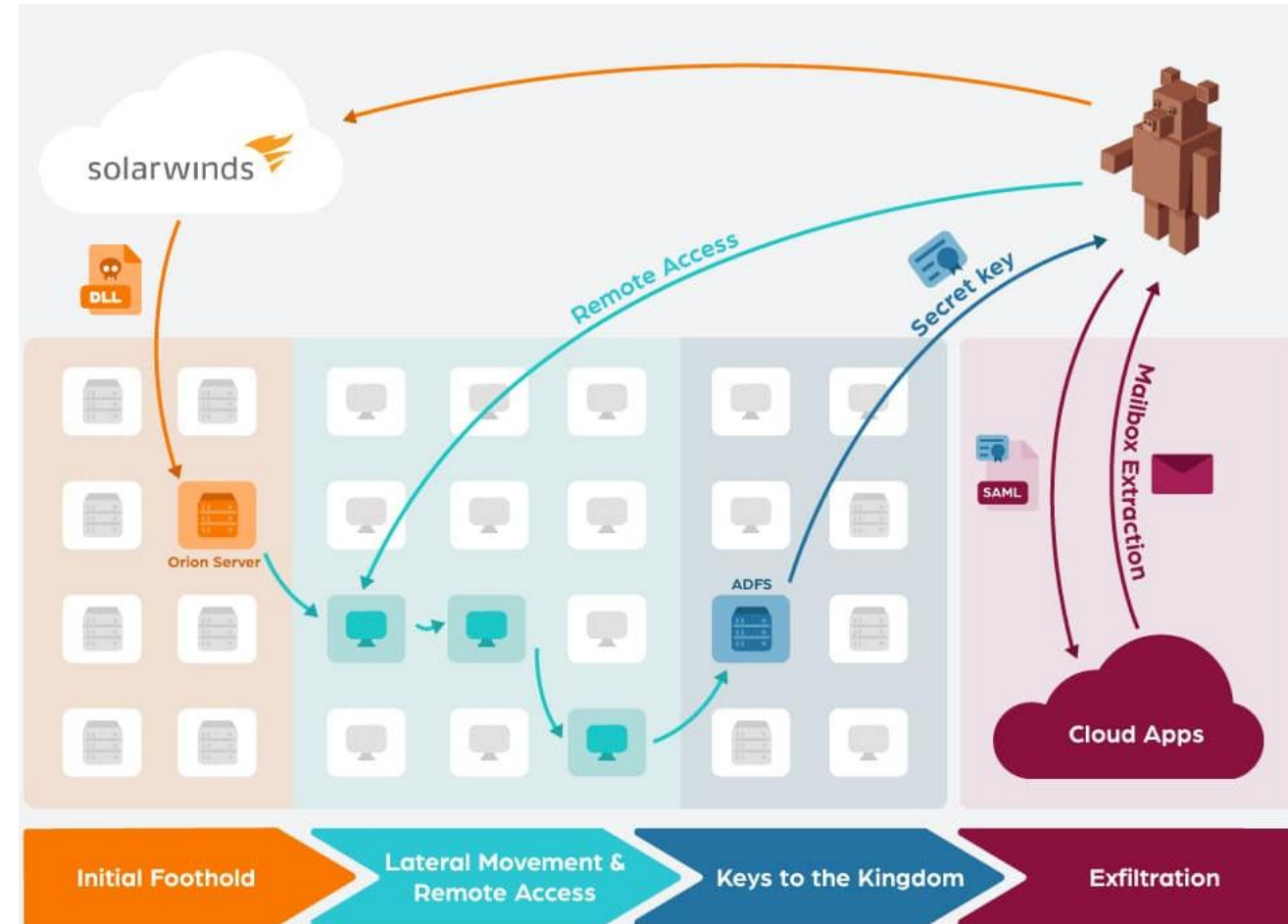
Protect everything

Asymmetric Challenge

Solar Winds: One of the biggest cyber attacks ever



Solar Winds: One of the biggest cyber attacks ever

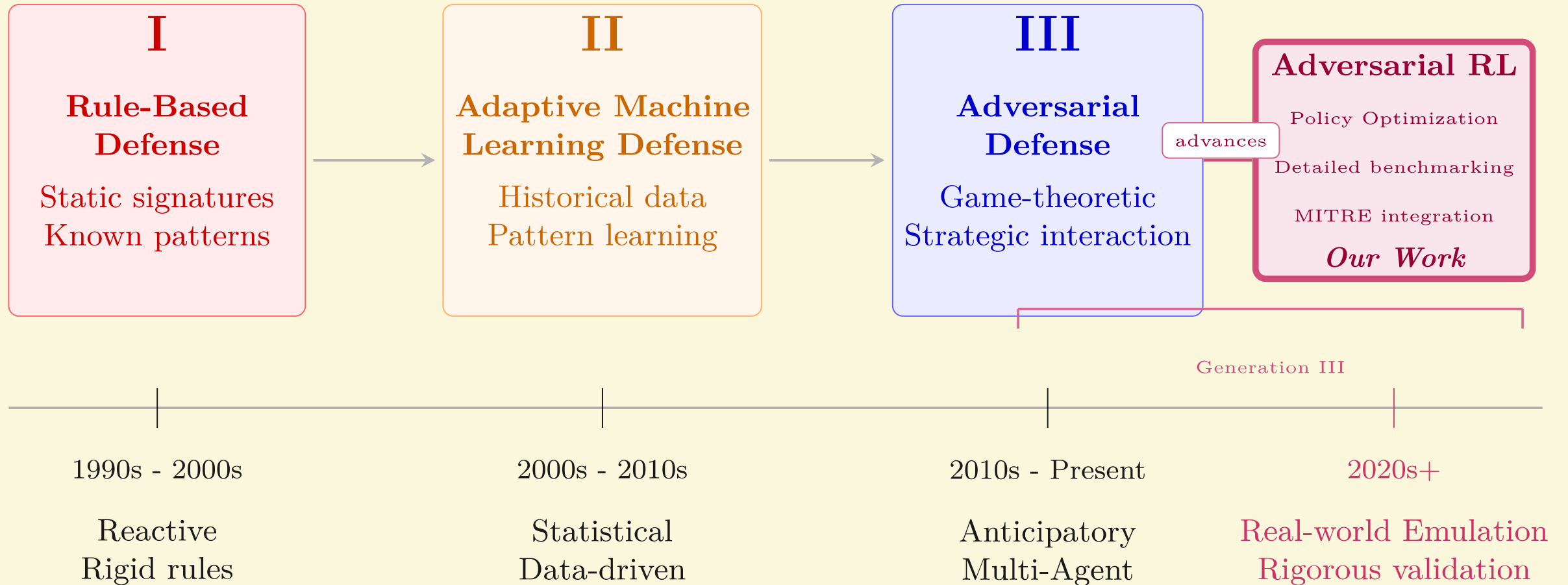


Our Solution

Adaptive RL-driven deception that creates uncertainty for attackers through learned strategic honeypot/decoy placement

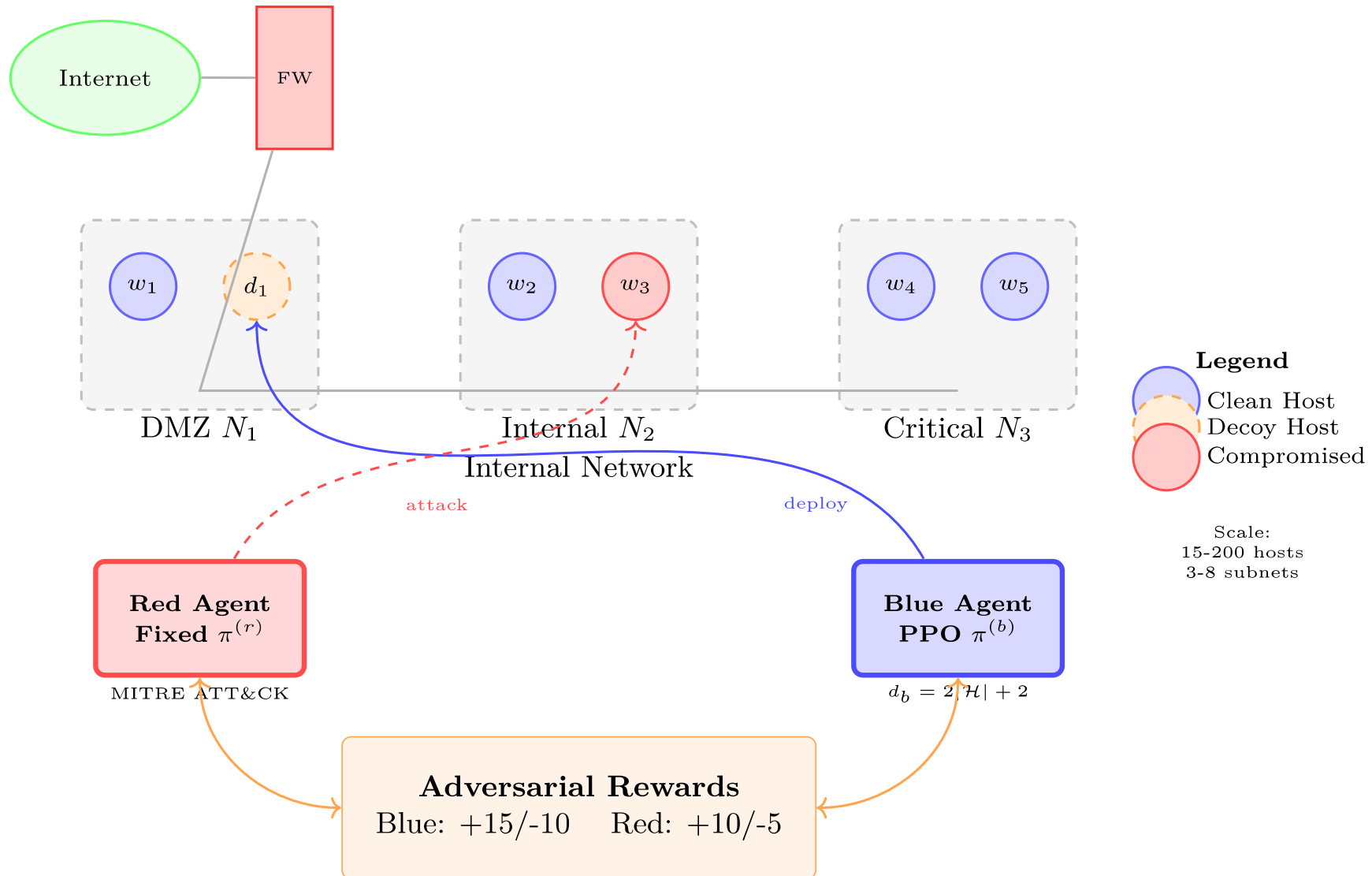
Evolution of Cyber Defence

Cyber Defence Through the Ages



Network and Agent Environment Overview

Interaction Dynamics



Key Notation

Notation	Definition
H	Episode length (timesteps), $H=50$
t	Timestep index
γ	Discount factor (0.99)
$\pi^{(b)}$	Blue policy (learned)
$\pi_{fixed}^{(r)}$	Red policy (fixed adversary)
$\mathcal{S}_t^{(b)}$	Blue observation/state vector at t
$\mathcal{A}_t^{(b)}$	Blue action at t
$R_t^{(b)}$	Blue reward at t
$J^{(b)}(\pi)$	Blue expected return objective
$\mathcal{A}^{(b)}$	Blue action space
d_b	Dimension of blue observation vector ($2 W +2$)

3.6 Environment Selection and Extensibility Rationale

Why Cyberwheel? The Cyberwheel framework was developed by Oak Ridge National Laboratory (ORNL) as a modular, extensible platform for cybersecurity research and was selected after comparative review of alternative cyber-range simulation environments (e.g., CybORG, CAGE, OpenAI Gym network extensions) based on three key advantages: (1) **native deception support** for decoy deployment state transitions crucial to our research, (2) **modular YAML-driven configuration** enabling systematic experimentation and baseline comparison, and (3) **scalable architecture** supporting networks from 15 hosts to enterprise scale with deterministic adversary modes for controlled learning analysis.

ORNL Repository and Development Context: The original Cyberwheel framework and documentation are maintained by Oak Ridge National Laboratory, with development motivated by the need for reproducible, scalable cybersecurity RL research platforms. This aligns directly with our requirements for systematic baseline comparison and experimental rigor.¹

Design Considerations: Our experimental design focuses on establishing fundamental defensive learning behaviors using a fixed adversary model. This approach

¹Cyberwheel open-source repository: <https://github.com/ORNL/cyberwheel>

Problem Formulation

We model cyber defense as a two-player game between an attacker (red agent) and defender (blue agent) operating on a network topology. Each game consists of up to $H = 50$ timesteps, representing a complete attack scenario.

Agent Roles: The red agent follows a fixed policy $\pi_{\text{fixed}}^{(r)}$ that executes MITRE ATT&CK techniques against network hosts. The blue agent learns a policy $\pi^{(b)}$ through PPO to strategically deploy decoys that mislead attackers while protecting real assets.

Turn-Based Interaction: At each timestep t , the blue agent first observes the network state $S_t^{(b)} \in \mathbb{R}^{d_b}$ and selects a defensive action $A_t^{(b)}$ (deploy decoy, remove decoy, or do nothing). The red agent then attacks a target, updating the network state for the next timestep.

Adversarial Rewards: When the red agent attacks real hosts, it gains positive reward while the blue agent receives negative reward. When the red agent attacks decoys, the rewards reverse: the blue agent receives a deception bonus ($+10 \times$ red reward) while the red agent is penalized.

Objective

Maximize Expected Defender Return

$$J^{(b)}(\pi^{(b)}) = \mathbb{E} \left[\sum_{t=0}^{H-1} \gamma^t R_t^{(b)} \right]$$

- Expectation over environment transitions and a fixed red policy; with discount factor $\gamma=0.99$.

Red Agent

$$\mathcal{S}_t^{(r)} = \{\text{current_host}_t, \text{history}_t, \text{tracked_hosts}_t, \text{unimpacted_hosts}_t, \text{services_map}_t\} \quad (6)$$

where $\text{current_host}_t \in \mathcal{W}$ tracks physical location, history_t maintains reconnaissance intelligence, $\text{tracked_hosts}_t \subseteq \mathcal{W}$ contains discovered hosts, $\text{unimpacted_hosts}_t$ tracks remaining targets, and services_map_t maps viable attack techniques per host.

The red agent's action selection follows a two-stage deterministic process: (1) **target selection** using a fixed strategy (e.g., ServerDowntime prioritization), then (2) **killchain progression** through MITRE ATT&CK techniques per target host.

The complete action set available to the red agent includes:

$$\mathcal{A}^{(r)} = \{\text{pingsweep}, \text{portscan}, \text{discovery}, \text{lateral-movement}, \text{privilege-escalation}, \text{impact}\}$$

Blue Agent

The blue agent's state space $S_t^{(b)} \in \mathbb{R}^{d_b}$ has a simplified observation structure:

$$S_t^{(b)} = \begin{bmatrix} \text{current_alerts}_t \\ \text{alert_history}_t \\ \text{metadata}_t \end{bmatrix} \quad (8)$$

where $\text{current_alerts}_t \in \{0, 1\}^{|\mathcal{W}|}$ represents immediate threat indicators per host, $\text{alert_history}_t \in \{0, 1\}^{|\mathcal{W}|}$ captures cumulative attack memory enabling pattern learning, and $\text{metadata}_t \in \mathbb{R}^2$ contains environmental constants: a fixed value of -1 and the total number of active decoys.

Blue Agent

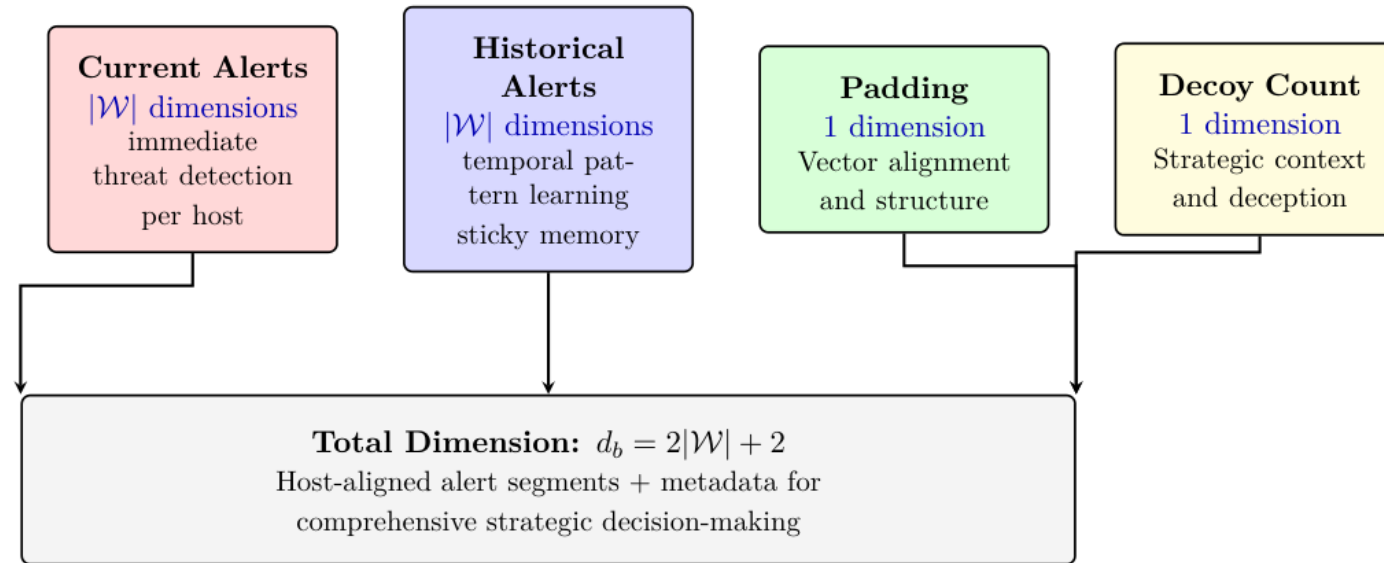
State Space Dimension

The blue agent's state space dimension is formally defined as:

$$d_b = 2|\mathcal{W}| + 2 \quad (9)$$

This compact representation efficiently captures all necessary defensive intelligence: alert components contribute $2|\mathcal{W}|$ dimensions (current + historical threat indicators per host), while metadata components add 2 dimensions (environmental constants: fixed value -1 and total decoy count).

This structure enables both immediate threat response and strategic pattern learning. Figure 4 provides a detailed breakdown of the observation vector structure and dimensionality.



3.4.2 Action Space and Defense Operations

The blue agent's action space consists of defensive operations implemented as a discrete action space:

$$\mathcal{A}^{(b)} = \mathcal{A}_{\text{deploy}} \cup \mathcal{A}_{\text{remove}} \cup \{\text{nothing}\} \quad (10)$$

where $\mathcal{A}_{\text{deploy}} = \{(\text{deploy_decoy}, n_j) : n_j \in \mathcal{N}\}$ deploys a decoy on subnet n_j , $\mathcal{A}_{\text{remove}} = \{(\text{remove_decoy}, n_j) : n_j \in \mathcal{N}\}$ removes a decoy from subnet n_j , and nothing represents no action (observation only).

Total action space size: $|\mathcal{A}^{(b)}| = 2|\mathcal{N}| + 1$.

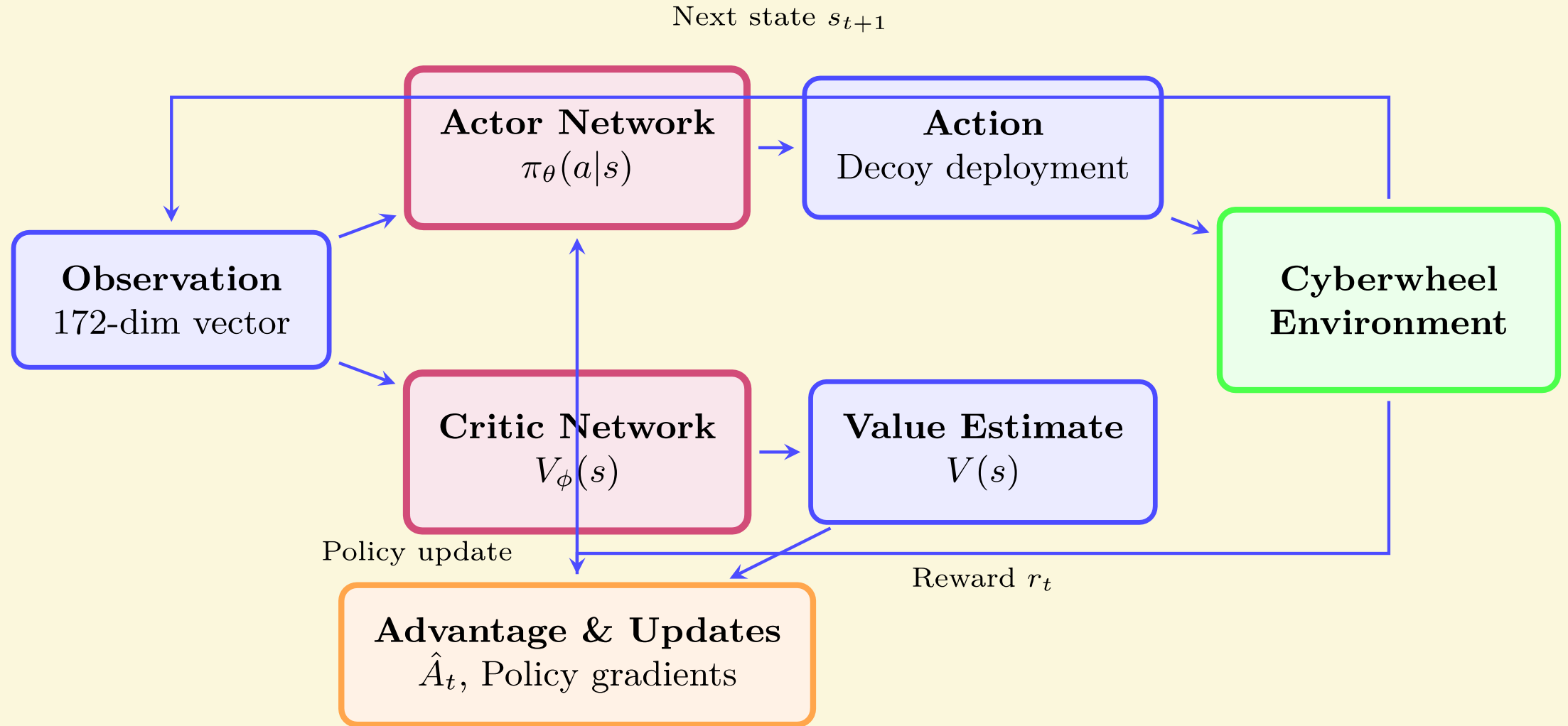
Reward Decomposition

Component	Value
Deception bonus	$+10 \times v_r$ (exp.)
Deploy cost	-20

- $R_t^{(b)} = R_{\text{attack}}^{(b)} + R_{\text{action}}^{(b)}$
- $R_{\text{attack}}^{(b)} = 10 \cdot v_r \cdot \mathbb{I}[\text{red attacks decoy}] - v_r \cdot \mathbb{I}[\text{red attacks asset}]$
- $R_{\text{action}}^{(b)} = -20 \cdot \mathbb{I}[a_t^{(b)} = \text{deploy}] - 1 \cdot \mathbb{I}[a_t^{(b)} = \text{remove}]$

Proximal Policy Optimization

Architecture Overview

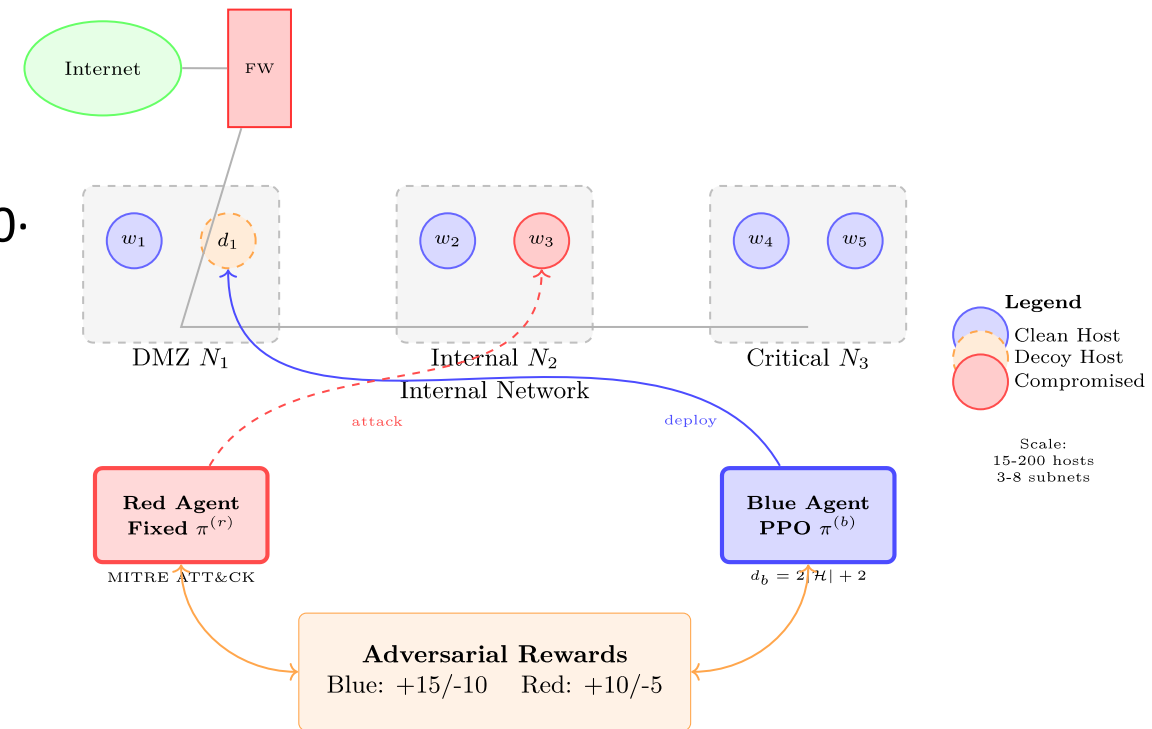


PPO Training for Blue Agent

```
1: Phase 1: Experience Collection
2: for  $t = 1$  to  $T$  do
3:   for  $h = 1$  to  $H$  do
4:     Observe state  $S_t^{(b)}$ 
5:     Sample action  $A_t^{(b)} \sim \pi_\theta(\cdot \mid S_t^{(b)})$ 
6:     Execute action, observe reward  $R_t^{(b)}$ 
7:     Store transition  $(S_t^{(b)}, A_t^{(b)}, R_t^{(b)}, S_{t+1}^{(b)})$ 
8:   end for
9: end for
10: Phase 2: Advantage Computation
11: Compute advantages  $\{\hat{A}_t\}$  using GAE
12: Compute returns  $\{R_t^{\text{total}}\}$ 
13: Phase 3: Policy Update
14: for  $k = 1$  to  $K$  epochs do
15:   for each minibatch in experience buffer do
16:     Compute policy loss:  $L^{\text{CLIP}}(\theta) = \max(-\hat{A}_t \cdot r_t(\theta), -\hat{A}_t \cdot \text{clip}(r_t(\theta), 1 - 0.2, 1 + 0.2))$ 
17:     Compute value loss:  $L^{\text{VF}}(\phi) = 0.5 \cdot (V_\phi(s_t) - \hat{R}_t)^2$  (with optional clipping)
18:     Compute entropy loss:  $L^{\text{ENT}} = -\sum_a \pi_\theta(a \mid s) \log \pi_\theta(a \mid s)$ 
19:     Total loss:  $L^{\text{total}} = L^{\text{CLIP}} - 0.01 \cdot L^{\text{ENT}} + 0.5 \cdot L^{\text{VF}}$ 
20:     Update:  $\theta \leftarrow \theta - \alpha \nabla_\theta L^{\text{total}}$ 
21:   end for
22: end for
```

Worked Example

- Step 0: Blue deploys decoy on subnet N1 (cost -20).
- Step 1: Red attacks host w3 (hits decoy). Reward: Blue +10; Red penalised.
- Step 2: Blue observes alert on decoy; updates policy context.
- Step 3: Blue removes decoy later (cost -1) or redeploys elsewhere; cumulative return changes accordingly.



Methodology

Methodology

Questions

How effectively can PPO agents learn optimal decoy deployment strategies in cybersecurity environments?

How does learned defensive behavior compare against static, rule-based, and inactive approaches?

What strategic behaviors emerge from RL-based defense, and how do they scale across network sizes?

Experimental Setup

- Multi-Agent Setup: Learning blue vs fixed red (295 techniques)
- Networks: 15-200 hosts, realistic enterprise topologies
- Baselines: Static, Rule-based, Inactive

Contributions

First systematic PPO validation for cyber deception

Real-world MITRE integration with +90% fidelity

Scalability characterization across enterprise networks

Strategic behavior analysis reveals emergent intelligence

Reproducible methodology with open protocols

Fixed Red Agent

```
1: Input: Current internal state, network knowledge
2: Extract phase  $\phi$  and position  $p$  from state
3: if  $\phi = \text{discovery}$  then
4:   Select network scanning action on current subnet
5:   if sufficient network topology discovered then
6:     Transition to reconnaissance phase
7:   end if
8: else if  $\phi = \text{reconnaissance}$  then
9:   Enumerate services and identify vulnerabilities
10:  if exploitable vulnerability found then
11:    Transition to privilege-escalation phase
12:  end if
13: else if  $\phi = \text{privilege-escalation}$  then
14:   Attempt lateral movement to high-value targets
15:   if critical server compromised then
16:     Transition to impact phase
17:   end if
18: else if  $\phi = \text{impact}$  then
19:   Execute data exfiltration and disruption actions
20: end if
21: return Action  $A_t^{(r)}$ 
```

MITRE ATT&CK

Initial Access	Execution	Persistence	Evasion	Discovery	Lateral Movement	Collection	Command and Control	Inhibit Response Function	Impair Process Control	Impact
Data Historian Compromise	Change Program State	Hooking	Exploitation for Evasion	Control Device Identification	Default Credentials	Automated Collection	Commonly Used Port	Activate Firmware Update Mode	Brute Force I/O	Damage to Property
Drive-by Compromise	Command-Line Interface	Module Firmware	Indicator Removal on Host	I/O Module Discovery	Exploitation of Remote Services	Data from Information Repositories	Connection Proxy	Alarm Suppression	Change Program State	Denial of Control
Engineering Workstation Compromise	Execution through API	Program Download	Masquerading	Network Connection Enumeration	External Remote Services	Detect Operating Mode	Standard Application Layer Protocol	Block Command Message	Masquerading	Denial of View
Exploit Public-Facing Application	Graphical User Interface	Project File Infection	Rogue Master Device	Network Service Scanning	Program Organization Units	Detect Program State		Block Reporting Message	Modify Control Logic	Loss of Availability
External Remote Services	Man in the Middle	System Firmware	Rootkit	Network Sniffing	Remote File Copy	I/O Image		Block Serial COM	Modify Parameter	Loss of Control
Internet Accessible Device	Program Organization Units	Valid Accounts	Spoof Reporting Message	Remote System Discovery	Valid Accounts	Location Identification		Data Destruction	Module Firmware	Loss of Productivity and Revenue
Replication Through Removable Media	Project File Infection		Utilize/Change Operating Mode	Serial Connection Enumeration		Monitor Process State		Denial of Service	Program Download	Loss of Safety
Spearphishing Attachment	Scripting					Point & Tag Identification		Device Restart/Shutdown	Rogue Master Device	Loss of View
Supply Chain Compromise	User Execution					Program Upload		Manipulate I/O Image	Service Stop	Manipulation of Control
Wireless Compromise						Role Identification		Modify Alarm Settings	Spoof Reporting Message	Manipulation of View
						Screen Capture		Modify Control Logic	Unauthorized Command Message	Theft of Operational Information
						Program Download				
						Rootkit				
						System Firmware				
						Utilize/Change Operating Mode				

Static Decoy Deployment

Require: Network subnets $\mathcal{N}$, placement interval $T_{\text{interval}} = 5$, max decoys $M_{\text{decoys}} = 5$

```
1: step_count  $\leftarrow$  0, deployed_count  $\leftarrow$  0
2: while simulation active do
3:   step_count  $\leftarrow$  step_count + 1
4:   if step_count mod  $T_{\text{interval}} \neq 0$  then
5:     return nothing {Wait for next interval}
6:   end if
7:   if deployed_count <  $M_{\text{decoys}}$  then
8:      $n_j \leftarrow \text{random\_choice}(\mathcal{N})$  {Random subnet selection}
9:     deploy_decoy( $n_j$ )
10:    deployed_count  $\leftarrow$  deployed_count + 1
11:  else
12:    remove_oldest_decoy() {Capacity management}
13:    deployed_count  $\leftarrow$  deployed_count - 1
14:  end if
15: end while
```

Rule Based Temporal Defence

Require: Time step t , deployed decoy count d , maximum decoys $d_{\max} = 6$

```
1: if  $t \leq 10$  AND  $d < 2$  then  
2:    $n_j \leftarrow \text{subnets}[d \bmod |\mathcal{N}|]$  {Round-robin subnet selection}  
3:    $\text{deploy\_decoy}(n_j)$   
4:    $d \leftarrow d + 1$   
5: else if  $t \bmod 15 = 0$  AND  $d < d_{\max}$  then  
6:    $n_j \leftarrow \text{random\_subnet}()$  {Random subnet selection}  
7:    $\text{deploy\_decoy}(n_j)$   
8:    $d \leftarrow d + 1$   
9: else if  $d \geq d_{\max}$  AND  $t \bmod 20 = 0$  then  
10:   $\text{remove\_decoy}()$  {Remove oldest decoy}  
11:   $d \leftarrow d - 1$   
12: else  
13:  return nothing {No action}  
14: end if
```

Inactive Control Baseline

Require: Time step t (ignored)
1: **return** nothing {Always take no action}

Results

Key Results

Algorithm Performance

Algorithm	Reward
PPO	309.14
Static Baseline	163.81
Rule Baseline	79.16
Inactive	-1.61

PPO Security Effectiveness:

80%+ improvement over static baseline 73.2% deception success rate (vs 45.1% static)

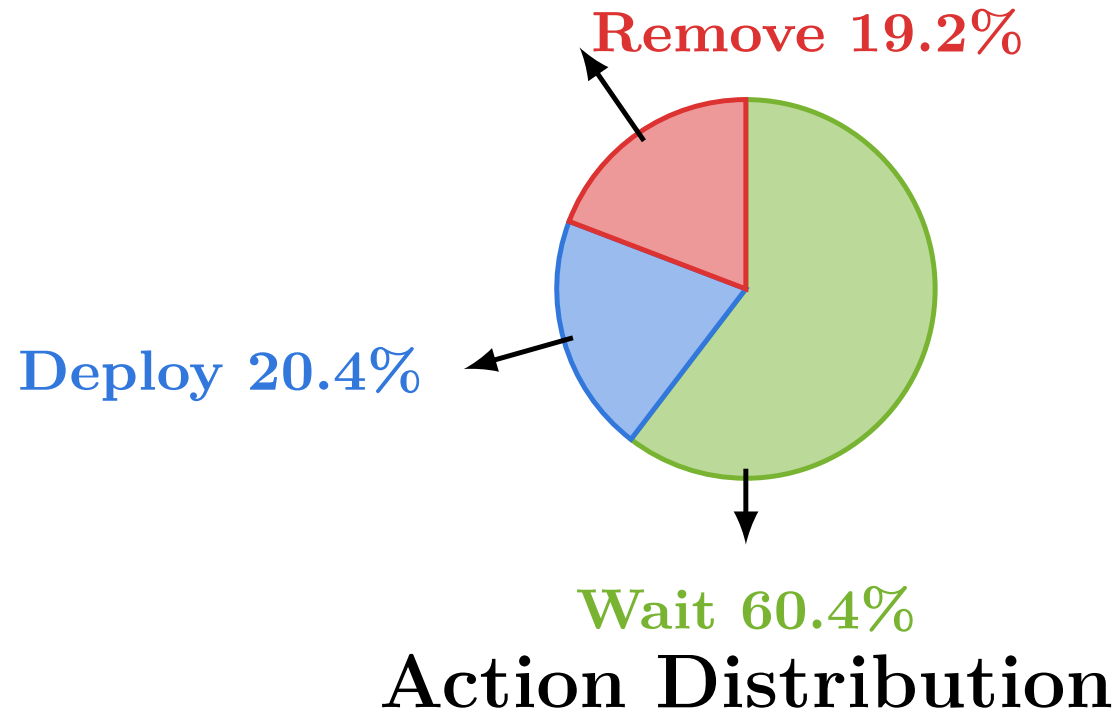
82.4% asset protection rate (vs 61.7% static)

2.3 timestep detection latency (vs 4.1 static)

92.3% correlation with MITRE ATT&CK Patterns

84.8% defense success against real attacks

Learned Behaviour



Insights

Agent learns when NOT to act (patience)

Balances resource costs vs security benefits

Adapts to attacker (Red Agent) patterns without explicit programming

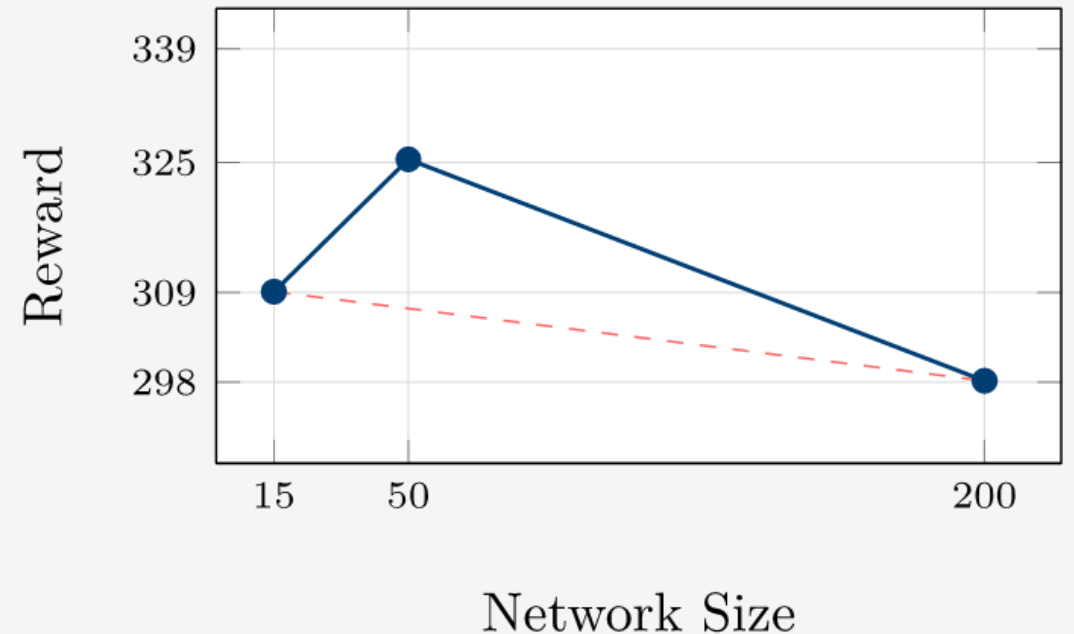
Intelligence

- Balanced Strategy: 20.4% deploy, 19.2% remove, 60.4% wait
- Learned Restraint: Avoids over-deployment traps
- Superior Consistency: $\sigma = 37.37$ vs baseline variance
- Adaptive Timing: Strategic waiting periods

Scalability Analysis

Performance by Size

Algorithm	Reward
15	309.14
50	325.42
200	298.17



Properties

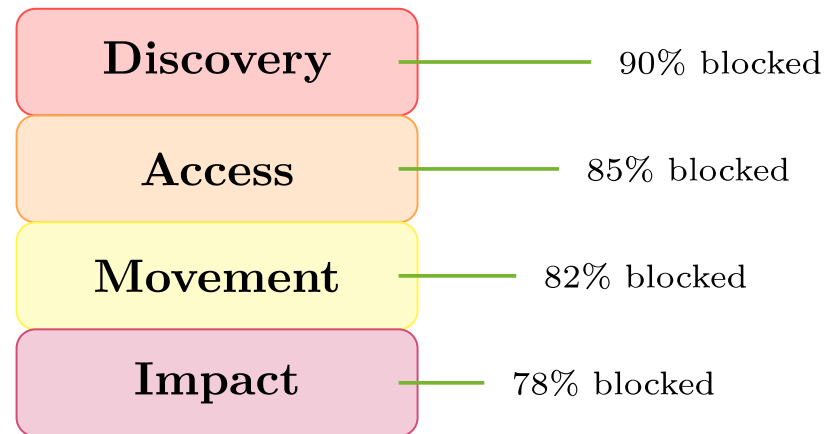
- **Consistent performance range:** ~298-325 reward
- **Linear memory usage:** Practical for enterprise use
- **Considerable consistency:** Behavior does not drift widely across scales

Real World Validation

Testing in The Wild

MITRE ATT&CK Integration:

- **295 attack techniques** implemented
- **Behavioral correlation** with real command patterns
- Complete kill chain coverage (4 phases) Atomic Red Team validation



Attack Phase Defense

Attack Defence Success

84.8% success rate against real attack commands

Early disruption: 2.3 timestep detection

Proactive defense: Uncertainty creation vs reactive response

Discussion

Future Directions & Impact

We establish reinforcement learning as a viable approach for automated cybersecurity defense.

Research Extensions

Real-world deployment in operational environments

Adversarial co-evolution with learning attackers (self-play)

Multi-defender coordination across distributed networks
Integration with existing security infrastructure (SIEM, EDR)

Explainable AI to aid analyst understanding and boost trust

Key Takeaways

- RL agent successfully learns sophisticated adaptive defense strategies
- 80%+ improvement demonstrates clear superiority
- Strategic restraint emerges naturally without explicit programming
- Performance scales robustly to enterprise networks
- Real attack correlation validates deployment viability

Limitations

Our significant contributions in adversarial cybersecurity RL has these limitations

- Simulation Environment Constraints
- Attack Model Scope
- Computational Resource Requirements
- Computational Resource Requirements
- Evaluation Timeframe
- Real-World Deployment Considerations

Conclusion

Conclusion

Performance

80%+ improvement over baselines

Intelligence

Strategic behavior emerges naturally

Scale

Consistent range up to 200 hosts

Impact

Rigorous validation of reinforcement learning for cybersecurity deception with real-world attack correlation and enterprise-scale deployment readiness

IMPERIAL

Thank You

Questions?

miracle.akanmode24@imperial.ac.uk